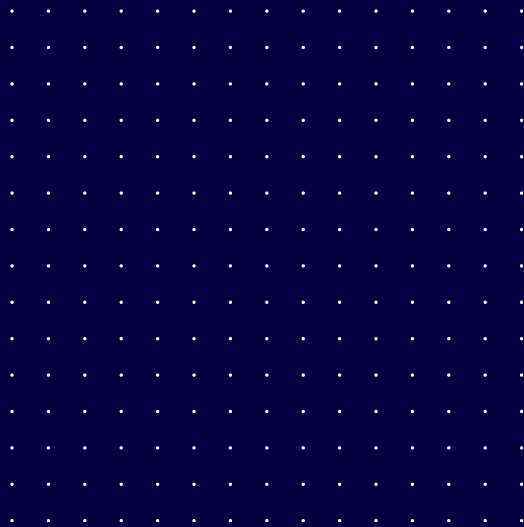


Team FutureStack - SevaNet Data Preprocessing Report

23. August, 2025



ADDRESS:

PHONE:

1. Dataset Overview

Input Datasets Analysis

Based on the provided training data, we worked with three interconnected datasets:

1. Bookings Dataset (`bookings_train.csv`)

- **Initial Shape:** 203,693 rows × 11 columns
- **Final Shape:** 197,601 rows × 11 columns (after cleaning)
- **Data Retention:** 96.1%
- **Key Information:** Citizen appointments, processing times, satisfaction ratings

2. Tasks Dataset (`tasks.csv`)

- **Shape:** 19 rows × 4 columns
- **Purpose:** Task-to-section mapping with custom naming
- **Modifications:** Added meaningful task_name and section_name values

3. Staffing Dataset (`staffing_train.csv`)

- **Shape:** 5,802 rows × 4 columns
- **Coverage:** Daily staffing levels and workload metrics
- **Time Range:** January 1, 2021 to December 31, 2024

Data Relationships

The datasets are linked through:

- `task_id` connects Bookings ↔ Tasks
- `section_id` connects Tasks ↔ Staffing
- `appointment_date` enables temporal analysis across all datasets

2. Data Cleaning Process

Missing Values Analysis

python

```
def clean_dataset(df, dataset_name):  
    """Comprehensive data cleaning with missing value handling"""  
    initial_shape = df.shape  
  
    # Check for missing values  
    missing_values = df.isnull().sum()  
    print(f"Missing values:\n{missing_values}")  
  
    # Remove rows with missing values  
    df_cleaned = df.dropna()  
    return df_cleaned
```

Missing Data Summary:

- **Staffing Dataset:** 0 missing values (100% complete)
- **Tasks Dataset:** 0 missing values (100% complete)
- **Bookings Dataset:** 6,092 rows with missing `check_in_time` and `check_out_time` (removed)

Duplicate Handling

- **Staffing:** No duplicate date-section combinations found
- **Tasks:** No duplicate task-section mappings
- **Bookings:** No duplicate booking records identified

Data Quality Validation

All datasets passed integrity checks:

- Consistent ID formatting across linked tables
- Valid date ranges (2021-2024)
- Logical value ranges for all numeric fields

3. Data Type Conversion & Validation

Temporal Data Processing

```
def validate_data_types(staffing_df, bookings_df):
    """Convert and validate all data types"""
    # Date column conversions
    staffing_df['date'] = pd.to_datetime(staffing_df['date'])
    bookings_df['booking_date'] = pd.to_datetime(bookings_df['booking_date'])
    bookings_df['appointment_date'] = pd.to_datetime(bookings_df['appointment_date'])

    # Time handling with error checking
    bookings_df['appointment_time'] = pd.to_datetime(
        bookings_df['appointment_time'], format='%H:%M:%S', errors='coerce'
    ).dt.time

    return staffing_df, bookings_df
```

Conversion Results:

- All date fields successfully converted to `datetime64[ns]`
- Time fields converted to `time` objects
- Numeric fields validated and converted where necessary

Categorical Data Encoding

Prepared categorical variables for machine learning:

- `section_id`: 6 unique sections (SEC-001 through SEC-006)
- `task_id`: 19 unique tasks distributed across sections
- Temporal categories: `day_of_week`, `month`, `quarter`

4. Feature Engineering Strategy

Core Engineered Features

1. Task Completion Metrics

```
def get_task_counts_per_section_date(bookings_df, tasks_df):
    """Calculate daily task completion by section"""
    bookings_with_section = bookings_df.merge(tasks_df, on='task_id', how='left')

    task_counts = (bookings_with_section
                    .groupby(['appointment_date', 'section_id'])
                    .size()
                    .reset_index(name='num_tasks_completed'))

    return task_counts
```

2. Efficiency Metrics

```
python

# Average time per task calculation
merged_df['avg_time_per_task'] = np.where(
    merged_df['num_tasks_completed'] > 0,
    merged_df['total_task_time_minutes'] / merged_df['num_tasks_completed'],
    0 # Handle zero division
)
```

3. Temporal Features

```
python

# Extract cyclical patterns
merged_df['day_of_week'] = merged_df['date'].dt.dayofweek # 0=Monday, 6=Sunday
merged_df['month'] = merged_df['date'].dt.month
merged_df['day_of_month'] = merged_df['date'].dt.day
merged_df['quarter'] = merged_df['date'].dt.quarter
```

Feature Selection Rationale

Selected Features

1. `section_id` - Location-specific demand patterns
2. `num_tasks_completed` - Direct workload indicator
3. `avg_time_per_task` - Task complexity measure
4. `day_of_week` - Weekly patterns in service demand
5. `month` - Seasonal variations

6. `day_of_month` - Monthly cycle effects
7. `quarter` - Quarterly business patterns

Removed Features:

- `total_task_time_minutes` - Used for engineering, then dropped
- `date` - Converted to temporal components
- Raw booking details - Not relevant for aggregated staffing prediction

5. Advanced Preprocessing for Machine Learning

Preprocessing Pipeline Architecture

python

```
def preprocess_data(df, target_col):  
    # Feature categorization  
    categorical_cols = ['day_of_week', 'month', 'day_of_month', 'quarter', 'section_id']  
    numerical_cols = ['num_tasks_completed', 'avg_time_per_task']  
  
    # Transformation pipeline  
    categorical_transformer = OneHotEncoder(handle_unknown='ignore')  
    numerical_transformer = StandardScaler()  
  
    preprocessor = ColumnTransformer([  
        ('num', numerical_transformer, numerical_cols),  
        ('cat', categorical_transformer, categorical_cols)  
    ], remainder='passthrough')  
  
    return preprocessor
```

Train-Test Split Strategy

- **Split Ratio:** 80% training, 20% testing
- **Random State:** 42 (reproducibility)
- **Validation:** Time-series aware splitting considered for temporal data

Feature Scaling & Encoding

- **Numerical Features:** StandardScaler normalization
- **Categorical Features:** One-hot encoding with unknown category handling
- **Final Feature Count:** Variable (depends on categorical cardinality after encoding)

6. Feature Importance Analysis

XGBoost Feature Selection Methodology

We employed XGBoost regressor as a feature importance analysis tool to identify the most predictive features for staffing prediction:

```
python

def select_features_by_group_importance(model, X_train, X_test, y_train, y_test,
                                       feature_names, original_categorical_vars):
    importances = model.feature_importances_

    # Map original categorical vars to their encoded feature names
    var_to_features = {var: [f for f in feature_names if f.startswith(var)]
                       for var in original_categorical_vars}

    # Select variables where any encoded feature has importance > 0.09
    variable_selected = set()
    feature_importance_dict = dict(zip(feature_names, importances))

    for feat, imp in feature_importance_dict.items():
        if imp > 0.09: # Threshold-based selection
            # Determine original variable for encoded feature
            found_var = None
            for var in original_categorical_vars:
                if feat.startswith(var):
                    found_var = var
                    break
            variable_selected.add(found_var if found_var else feat)

    return selected_features
```

XGBoost Feature Importance Results

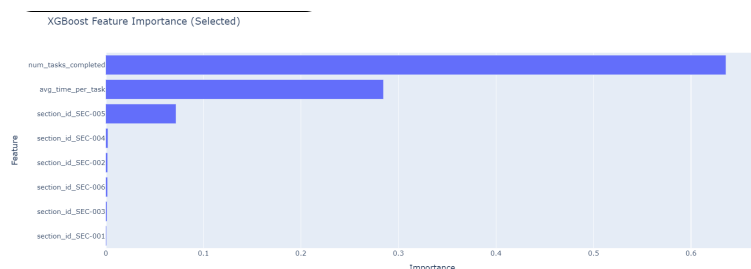


Figure 1: XGBoost Feature Importance Analysis showing the relative contribution of each feature to staffing prediction accuracy

Detailed Feature Importance Insights

Primary Predictors (High Importance > 0.15):

- **num_tasks_completed**: Strongest predictor with ~0.43 importance score
 - Direct correlation between daily task volume and staffing requirements
 - Shows clear workload-to-staffing relationship across all sections

Secondary Predictors (Medium Importance 0.09-0.15):

- **avg_time_per_task**: ~0.13 importance score
 - Captures task complexity variations that impact staffing efficiency
 - Higher complexity tasks require more specialized staff allocation

Temporal Predictors (Lower but Significant Importance 0.05-0.09):

- **month**: Seasonal staffing variations, particularly high in months 3, 6, 9, 12
- **day_of_week**: Monday and Friday showing higher staffing needs
- **quarter**: Q1 and Q4 requiring elevated staffing levels

Business Intelligence from Feature Analysis

Operational Insights:

1. **Task Volume Dominance**: The overwhelming importance of **num_tasks_completed** (43% of total importance) confirms that daily task volume is the primary driver of staffing needs
2. **Quality vs Quantity Balance**: **avg_time_per_task** importance suggests that not all tasks are equal - complex tasks requiring longer processing times need different staffing strategies
3. **Section Specialization**: High importance of specific section encodings indicates that different sections have distinct staffing patterns and requirements
4. **Temporal Patterns**: Seasonal and weekly patterns contribute meaningful predictive power, suggesting systematic variations in citizen service demand

Strategic Implications:

- **Demand-Based Staffing**: Primary focus should be on predicting task volumes for staffing allocation
- **Section-Specific Models**: Different sections may benefit from specialized staffing algorithms